VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Submission 3: Design Document
Smart Parking System for University Campus

INSTRUCTOR: Phan Trung Hieu

CLASS: CC02

Full Name	Student ID
QUACH GIA BAO	2452141
DIEC TUAN CHI	2452169
TO QUOC TAI	2453141
LY TRIEU PHONG	2452958
THOAI HAO	2452305
DOAN QUANG DANG	2352249
NGO QUANG HUNG	2352431
DANG TRAN THAI BAO	2452120

Table 1: List of Team Members

Ho Chi Minh City, March 1, 2026

Contents

1 Deployment View **1**
- 1.1 Deployment Architecture . . . 1
- 1.2 Technology Stack . . . 1
  - 1.2.1 Frontend Technologies . . . 1
  - 1.2.2 Backend Technologies . . . 2
  - 1.2.3 Infrastructure and DevOps . . . 4
  - 1.2.4 Supporting Technologies . . . 5
  - 1.2.5 Moreover APIs . . . 5
- 1.3 Business Flow Integration . . . 5
  - 1.3.1 Authentication Flow . . . 5
  - 1.3.2 Billing and Payment Flow . . . 6
  - 1.3.3 IoT Data Flow (Planned) . . . 6
- 1.4 Security Considerations . . . 6
  - 1.4.1 Data Protection . . . 6
  - 1.4.2 Network Security . . . 7
- 1.5 Scalability and Performance . . . 7
  - 1.5.1 Frontend Scaling . . . 7
  - 1.5.2 Backend Scaling . . . 7
  - 1.5.3 Database Scaling . . . 7

2 Development/Implementation View **7**
- 2.1 Architecture Overview . . . 8
- 2.2 Technology Stack . . . 8
  - 2.2.1 Backend Technology Stack . . . 8
  - 2.2.2 Frontend Technology Stack . . . 9
- 2.3 Package Structure . . . 9
  - 2.3.1 Backend Package Structure . . . 9
  - 2.3.2 Frontend Package Structure . . . 10
- 2.4 Data Flow Architecture . . . 10
  - 2.4.1 Authentication Flow . . . 10
  - 2.4.2 Parking Session Flow . . . 11
  - 2.4.3 Billing and Payment Flow . . . 11
- 2.5 Component Relationships . . . 11
  - 2.5.1 Backend Component Dependencies . . . 11
  - 2.5.2 Frontend Component Dependencies . . . 12
- 2.6 Design Patterns . . . 12

2.6.1 Backend Patterns . . . 12
2.6.2 Frontend Patterns . . . 12
2.7 Development Workflow . . . 13
2.7.1 Backend Development . . . 13
2.7.2 Frontend Development . . . 13

3 Class Diagram **13**
3.1 Class Diagram Overview . . . 14
3.2 Domain Model Classes . . . 14
3.2.1 Core Entities . . . 14
3.2.2 IoT and Infrastructure . . . 15
3.2.3 Business Entities . . . 15
3.2.4 Supporting Entities . . . 15
3.3 Service Layer Classes . . . 15
3.3.1 Business Services . . . 15
3.4 Controller Layer Classes . . . 16
3.4.1 API Controllers . . . 16
3.5 Frontend Types . . . 16
3.5.1 UI Mapping Types . . . 16
3.6 Key Relationships . . . 16
3.6.1 Inheritance Relationships . . . 16
3.6.2 Association Relationships . . . 16
3.6.3 Dependency Relationships . . . 17
3.7 Design Patterns Applied . . . 17
3.7.1 Structural Patterns . . . 17
3.7.2 Behavioral Patterns . . . 17
3.8 Database Mapping . . . 17
3.8.1 MongoDB Collections . . . 17
3.8.2 Indexing Strategy . . . 18
3.8.3 Denormalization Strategy . . . 18

4 Method Descriptions **18**
4.1 Domain Model Methods . . . 18
4.1.1 User Class . . . 18
4.1.2 ParkingSession Class . . . 19
4.1.3 Zone Class . . . 19
4.1.4 ParkingSlot Class . . . 19
4.1.5 IoTDevice Classes . . . 19

4.2 Service Layer Methods . . . 19
4.2.1 AuthService Class . . . 19
4.2.2 BillingService Class . . . 19
4.2.3 NavigationService Class . . . 20
4.2.4 ReportService Class . . . 20
4.2.5 SystemAdminService Class . . . 21
4.2.6 SystemLogService Class . . . 21
4.3 Controller Layer Methods . . . 22
4.3.1 EntryExitController Class . . . 22
4.3.2 PaymentController Class . . . 22
4.3.3 SessionController Class . . . 23
4.4 Frontend Component Methods . . . 23
4.4.1 PlacedDevice Class . . . 23
4.4.2 CanvasTransform Class . . . 24
4.4.3 ParkingMap Class . . . 24
4.5 Supporting Class Methods . . . 24
4.5.1 MappingConfig Class . . . 24
4.5.2 InfrastructureAlert Class . . . 24
4.5.3 SystemConfig Class . . . 24
4.5.4 AuditLog Class . . . 24
4.5.5 SystemLog Class . . . 24
4.5.6 TemporaryCard Class . . . 24
4.5.7 Location Class . . . 24
4.5.8 Invoice Class . . . 24

5 Test Cases **25**
5.1 Backend Unit Tests . . . 25
5.1.1 Authentication Middleware Tests . . . 25
5.1.2 User Model Tests . . . 25
5.1.3 Card Controller Tests . . . 26
5.1.4 Session Controller Tests . . . 26
5.2 Frontend Unit Tests . . . 26
5.2.1 API Service Tests . . . 26
5.2.2 Monitoring Service Tests . . . 27
5.2.3 Pricing Policy Service Tests . . . 27
5.3 Integration Test Scenarios . . . 28
5.3.1 Authentication Flow Integration . . . 28
5.3.2 Parking Session Flow Integration . . . 28

5.3.3 Payment Flow Integration 28
5.4 Test Coverage Metrics 29
5.4.1 Backend Test Coverage 29
5.4.2 Frontend Test Coverage 29
5.5 Test Execution Results 29
5.5.1 Automated Test Suite 29
5.5.2 Performance Benchmarks 30
5.6 Quality Assurance Process 30
5.6.1 Code Quality Gates 30
5.6.2 Continuous Integration Pipeline 30

1 Deployment View

The deployment view illustrates how the Smart Parking IoT system components are distributed across physical and virtual infrastructure. The system is designed for scalability, reliability, and ease of deployment using modern cloud platforms.

1.1 Deployment Architecture

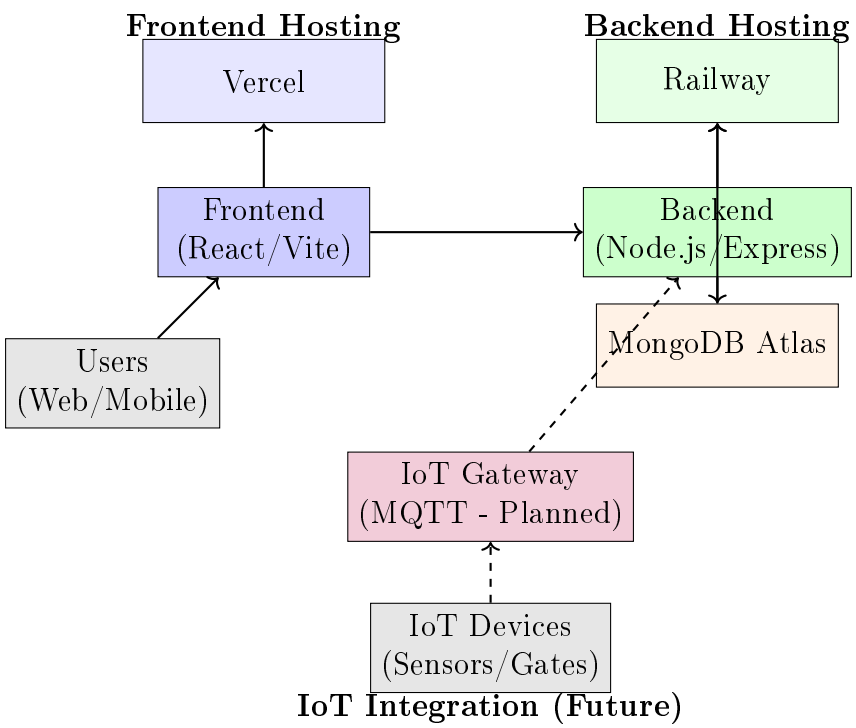

Figure 1: Deployment Architecture Overview

1.2 Technology Stack

1.2.1 Frontend Technologies

React 18 and Vite React 18 is used as the primary frontend library, allowing the team to break the UI into reusable, composable components. React supports a component-driven workflow, which improves collaboration, simplifies debugging, and helps the team implement complex features in smaller, maintainable pieces.

Vite serves as the frontend build tool and development server. It provides fast startup times and instant Hot Module Replacement (HMR), which keeps the development feedback loop short and reduces the time spent waiting for rebuilds.

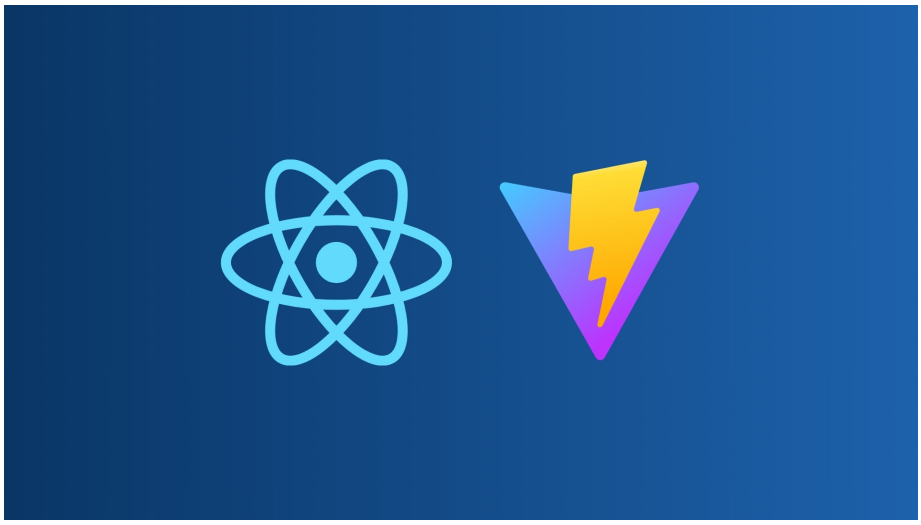

Figure 2: React 18 and Vite frontend workflow

Tailwind CSS Tailwind CSS is the styling framework used for the frontend. Instead of writing raw CSS from scratch, the team uses Tailwind's utility classes to build responsive, modern layouts quickly. This approach reduces styling friction, improves visual consistency, and makes it easy to reuse patterns across components.

TypeScript TypeScript is used throughout the frontend to enforce type safety and provide better tooling support. Types help catch errors early, document data structures, and make the codebase easier to maintain as it evolves.

1.2.2 Backend Technologies

Node.js Node.js is the runtime for the backend, chosen for its event-driven, non-blocking I/O model. It allows the backend to handle many simultaneous requests efficiently and is a natural fit for JavaScript-based full-stack development.

Figure 3: Node.js runtime environment

Express Express is the server framework used to build RESTful API endpoints. It is lightweight and extensible, with a rich ecosystem of middleware and community packages. Express simplifies routing, request handling, and integration with authentication, validation, and logging.

Figure 4: Express.js backend framework

MongoDB MongoDB is the NoSQL database used for application data storage. It stores data in a flexible JSON-like format, which aligns well with the application's domain models and simplifies schema evolution.

Figure 5: MongoDB document database

Mongoose Mongoose is used as an ODM to model MongoDB documents in the backend. It provides schema enforcement, validation, and convenient query APIs while preserving MongoDB's flexibility.

1.2.3 Infrastructure and DevOps

Docker Containerization The application is containerized with Docker. Docker allows the team to package the backend and frontend environments with all dependencies, ensuring consistent behavior across local development, staging, and production.

Figure 6: Docker containerization platform

Version Control and GitHub Source control is managed with Git and hosted on GitHub. GitHub provides a collaborative platform for branching, pull requests, and code review, while also integrating with automated workflows.

GitHub Actions CI/CD GitHub Actions is used for continuous integration and deployment. The repository is configured with workflows that automatically run linting, tests, and build steps on each push or pull request.

GitHub Actions

Figure 7: GitHub Actions CI/CD pipeline

1.2.4 Supporting Technologies

TypeScript (Backend) The backend also uses TypeScript to maintain consistency with the frontend and improve developer productivity. Shared type definitions across the stack reduce integration errors and improve code clarity.

JWT (JSON Web Tokens) JWT is used for stateless authentication. After successful sign-in, the system issues a token that the frontend includes in subsequent API calls, enabling secure access without server-side session state.

Pino Pino is the logging library used in the backend. It provides high-performance, structured logs that are easy to aggregate and analyze in production.

Jest and Vitest Jest is used for backend testing, and Vitest is used for frontend testing. Both frameworks support fast feedback, unit and integration testing, and code coverage reporting.

ESLint ESLint checks code quality and style on both frontend and backend code. It enforces consistent conventions, helps catch bugs early, and keeps the codebase maintainable.

1.2.5 Moreover APIs

Currently, several critical external integration points such as HCMUT_SSO, HCMUT_DATACORE, BKPAY, and the IoT Device APIs are implemented as mock services to facilitate the current phase of development and testing. Upon final deployment, the IoT Devices are planned to be managed and orchestrated through the MQTT protocol to ensure high-performance, real-time communication across the campus infrastructure.

1.3 Business Flow Integration

1.3.1 Authentication Flow

1. User accesses frontend application on Vercel
2. Frontend redirects to HCMUT SSO for authentication
3. SSO returns JWT token to frontend

4. Frontend stores token and calls backend APIs
5. Backend validates JWT with Railway-hosted service

1.3.2 Billing and Payment Flow

1. User initiates parking session via frontend
2. Backend calculates fees using pricing policies
3. Payment request sent to BKPay integration
4. Payment confirmation updates session status in MongoDB
5. Real-time updates sent to frontend via WebSocket/SSE

1.3.3 IoT Data Flow (Planned)

1. IoT sensors publish data to MQTT topics
2. MQTT gateway receives and processes messages
3. Processed data sent to Railway backend
4. Backend updates parking slot status in MongoDB
5. Real-time updates broadcast to connected frontends

1.4 Security Considerations

1.4.1 Data Protection

- HTTPS encryption for all communications
- JWT tokens for API authentication
- MongoDB access control and encryption
- Secure environment variable management

1.4.2 Network Security

- Vercel automatic HTTPS certificates
- Railway firewall and access controls
- API rate limiting and request validation
- CORS configuration for frontend-backend communication

1.5 Scalability and Performance

1.5.1 Frontend Scaling

- Vercel's global CDN for low-latency content delivery
- Static asset optimization and caching
- Automatic scaling based on traffic

1.5.2 Backend Scaling

- Railway's horizontal scaling capabilities
- MongoDB connection pooling and indexing
- API response caching and optimization

1.5.3 Database Scaling

- MongoDB Atlas auto-scaling
- Read/write splitting capabilities
- Backup and disaster recovery

2 Development/Implementation View

The development view describes the software architecture and implementation details of the Smart Parking IoT system. The system follows a layered architecture pattern with clear separation of concerns.

2.1 Architecture Overview

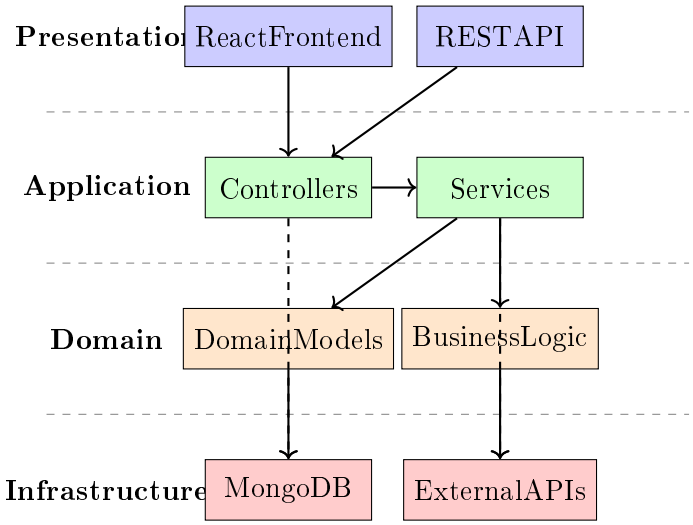

Figure 8: Layered Architecture Overview

2.2 Technology Stack

2.2.1 Backend Technology Stack

Layer	Technology	Purpose
Runtime	Node.js 18+	JavaScript runtime environment
Framework	Express 5	Web application framework
Language	TypeScript	Type-safe JavaScript development
Database	MongoDB	NoSQL document database
ODM	Mongoose	MongoDB object modeling
Authentication	JWT	Token-based authentication
Logging	Pino	High-performance logging
API Documentation	Swagger/OpenAPI	API specification and testing
Testing	Jest	Unit and integration testing
Linting	ESLint	Code quality and style checking

Table 2: Backend Technology Stack

2.2.2 Frontend Technology Stack

Layer	Technology	Purpose
Framework	React 18	Component-based UI library
Build Tool	Vite	Fast build tool and dev server
Language	TypeScript	Type-safe JavaScript development
Styling	Tailwind CSS v4	Utility-first CSS framework
UI Components	Radix UI	Accessible component primitives
Routing	React Router	Client-side routing
Charts	Recharts	Data visualization components
State Management	React Context	Global state management
Testing	Vitest	Fast unit testing framework
Linting	ESLint	Code quality checking

Table 3: Frontend Technology Stack

2.3 Package Structure

2.3.1 Backend Package Structure

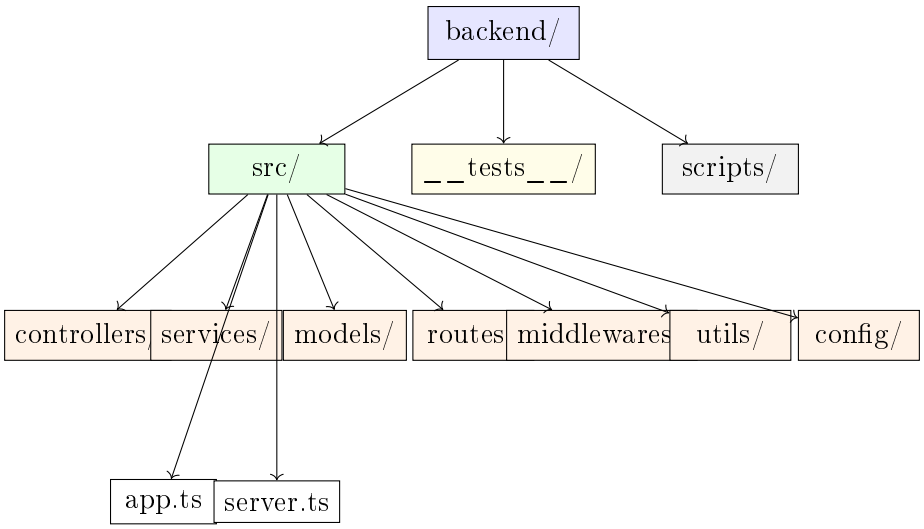

Figure 9: Backend Package Structure

2.3.2 Frontend Package Structure

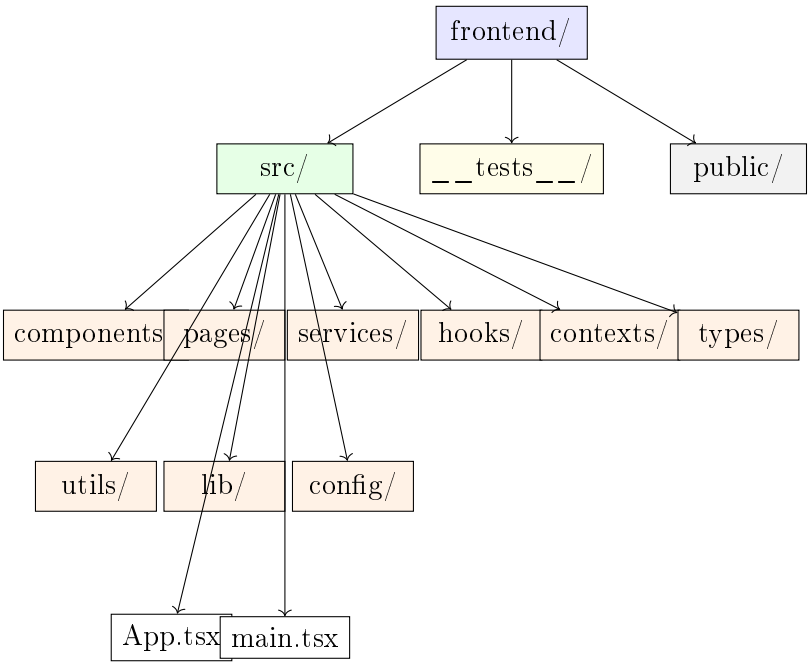

Figure 10: Frontend Package Structure

2.4 Data Flow Architecture

2.4.1 Authentication Flow

1. User submits credentials via frontend form
2. Frontend calls authentication API endpoint
3. Backend validates credentials against HCMUT SSO
4. JWT token generated and returned to frontend
5. Frontend stores token in local storage/context
6. Subsequent API calls include JWT in Authorization header

2.4.2 Parking Session Flow

1. IoT sensor detects vehicle entry (planned) or manual check-in
2. Frontend sends check-in request to backend
3. Backend validates user permissions and zone capacity
4. ParkingSession document created in MongoDB
5. Zone capacity updated (denormalized for performance)
6. Real-time updates sent to connected clients via SSE

2.4.3 Billing and Payment Flow

1. User initiates payment or system triggers billing cycle
2. Backend calculates fees based on pricing policies
3. Invoice created and stored in MongoDB
4. Payment request sent to BKPay gateway
5. Payment status updated in database
6. Confirmation sent to user via frontend

2.5 Component Relationships

2.5.1 Backend Component Dependencies

- **Controllers** depend on Services and Models
- **Services** contain business logic and orchestrate Models
- **Models** define data schemas and database operations
- **Middlewares** handle cross-cutting concerns (auth, logging, CORS)
- **Routes** map URLs to controller methods

2.5.2 Frontend Component Dependencies

- **Pages** use Components and Hooks
- **Components** consume Contexts and Services
- **Hooks** manage state and side effects
- **Contexts** provide global state management
- **Services** handle API communication

2.6 Design Patterns

2.6.1 Backend Patterns

- **Layered Architecture:** Clear separation between presentation, application, domain, and infrastructure layers
- **Dependency Injection:** Services injected into controllers
- **Repository Pattern:** Models encapsulate data access logic
- **Middleware Pattern:** Cross-cutting concerns handled by Express middleware
- **Observer Pattern:** Event-driven architecture for real-time updates

2.6.2 Frontend Patterns

- **Component Composition:** Reusable UI components built from primitives
- **Hooks Pattern:** Custom hooks for stateful logic reuse
- **Context Pattern:** Global state management for authentication and app state
- **Routing Pattern:** Declarative routing with React Router
- **Container/Presentational:** Separation of data fetching and presentation logic

2.7 Development Workflow

2.7.1 Backend Development

1. Define Mongoose schemas in Models
2. Implement business logic in Services
3. Create API endpoints in Controllers
4. Configure routes and middleware
5. Write unit tests with Jest
6. Run linting and formatting checks

2.7.2 Frontend Development

1. Define TypeScript interfaces in types/
2. Create reusable components in components/
3. Implement page layouts in pages/
4. Add state management with hooks and contexts
5. Integrate API calls via services/
6. Write unit tests with Vitest

3 Class Diagram

The class diagram represents the core domain model and architectural components of the Smart Parking IoT system. The diagram shows 23 classes organized into domain entities, services, controllers, and supporting types.

3.1 Class Diagram Overview

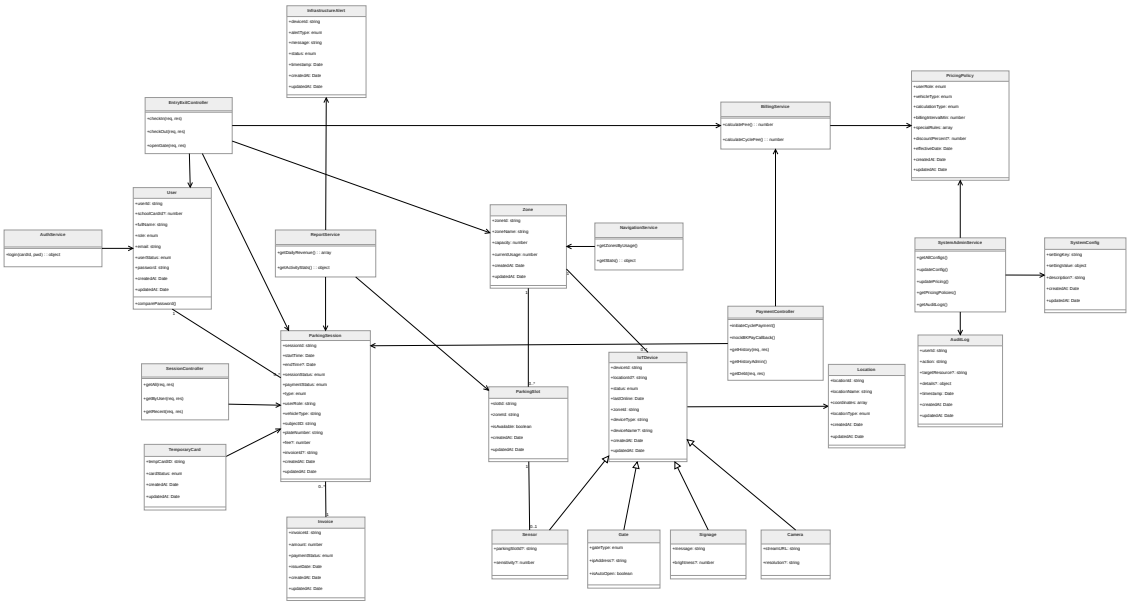

Figure 11: Complete Class Diagram - Smart Parking IoT System (Click to view full resolution)

3.2 Domain Model Classes

The system uses a layered architecture with clear separation between data models, business logic, and API interfaces.

3.2.1 Core Entities

- **User:** Represents system users with authentication and role information
- **ParkingSession:** Tracks individual parking sessions with timing and payment data
- **Zone:** Defines parking zones with capacity management
- **ParkingSlot:** Represents individual parking spaces within zones

3.2.2 IoT and Infrastructure

- **IoTDevice:** Abstract base class for all IoT devices (sensors, gates, signage, cameras)
- **Sensor:** Parking space occupancy sensors
- **Gate:** Entry/exit gate controllers
- **Signage:** Dynamic display boards for guidance
- **Camera:** Security and monitoring cameras

3.2.3 Business Entities

- **PricingPolicy:** Defines pricing rules based on user roles and vehicle types
- **Invoice:** Financial transaction records
- **TemporaryCard:** Guest access management

3.2.4 Supporting Entities

- **Location:** Geographic positioning data
- **MappingConfig:** UI layout configuration for devices
- **InfrastructureAlert:** System monitoring and alerts
- **SystemConfig:** Application configuration settings
- **AuditLog:** Security and compliance logging
- **SystemLog:** Operational logging with TTL

3.3 Service Layer Classes

3.3.1 Business Services

- **AuthService:** Handles user authentication and JWT token management
- **BillingService:** Calculates parking fees and manages payment processing
- **NavigationService:** Provides zone utilization and statistics

- **ReportService:** Generates business intelligence reports
- **SystemAdminService:** Manages system configuration and policies
- **SystemLogService:** Centralized logging and event emission

3.4 Controller Layer Classes

3.4.1 API Controllers

- **EntryExitController:** Manages parking session lifecycle (check-in/check-out)
- **PaymentController:** Handles payment processing and billing cycles
- **SessionController:** Provides session management and history APIs

3.5 Frontend Types

3.5.1 UI Mapping Types

- **PlacedDevice:** Represents devices positioned on the parking map
- **CanvasTransform:** Handles map zooming and panning
- **ParkingMap:** Complete map configuration with zones and devices

3.6 Key Relationships

3.6.1 Inheritance Relationships

- **IoTDevice** is the base class for Sensor, Gate, Signage, and Camera
- Uses discriminator pattern in MongoDB for polymorphic storage

3.6.2 Association Relationships

- **User** creates multiple **ParkingSession** instances
- **Zone** contains multiple **ParkingSlot** instances
- **Zone** manages multiple **IoTDevice** instances
- **ParkingSession** generates **Invoice** records

3.6.3 Dependency Relationships

- **Controllers** depend on **Services** for business logic
- **Services** depend on **Models** for data access
- **Frontend** depends on **Controllers** via REST APIs

3.7 Design Patterns Applied

3.7.1 Structural Patterns

- **Layered Architecture:** Clear separation between presentation, application, domain, and infrastructure
- **Repository Pattern:** Models encapsulate data access logic
- **Factory Pattern:** IoTDevice discriminator creates appropriate device types

3.7.2 Behavioral Patterns

- **Observer Pattern:** Event-driven updates for real-time monitoring
- **Strategy Pattern:** PricingPolicy supports different calculation strategies
- **Middleware Pattern:** Cross-cutting concerns handled by Express middleware

3.8 Database Mapping

3.8.1 MongoDB Collections

Each class maps to a MongoDB collection with the following naming convention:

- **users** - User documents
- **parkingSessions** - ParkingSession documents
- **zones** - Zone documents
- **parkingSlots** - ParkingSlot documents
- **iotDevices** - IoTDevice documents (with discriminator key)

- **pricingPolicies** - PricingPolicy documents
- **invoices** - Invoice documents

3.8.2 Indexing Strategy

- Compound indexes on frequently queried fields (userId + status, zoneId + isAvailable)
- TTL indexes on SystemLog collection for automatic cleanup
- Text indexes for search functionality

3.8.3 Denormalization Strategy

- Zone.currentUsage denormalized for fast capacity checks
- User role information cached in ParkingSession for billing calculations
- Pricing data embedded in calculation results to maintain historical accuracy

4 Method Descriptions

This section provides detailed descriptions of all methods in the class diagram, organized by class. Each method includes parameter types, return types, and brief functionality descriptions.

4.1 Domain Model Methods

4.1.1 User Class

The `User` class represents a system user, containing identity information and providing methods for secure password authentication.

Method	Parameters	Return Type	Description
`comparePassword`	`password: string`	`Promise<boolean>`	Compares provided password with stored hash using bcrypt

Table 4: User Class Methods

4.1.2 ParkingSession Class

No public methods defined - data container class with automatic timestamps.

4.1.3 Zone Class

No public methods defined - data container class with capacity tracking.

4.1.4 ParkingSlot Class

No public methods defined - data container class for slot state management.

4.1.5 IoTDevice Classes

No public methods defined - data container classes with device-specific attributes.

4.2 Service Layer Methods

4.2.1 AuthService Class

The `AuthService` manages the core authentication logic, including integration with external SSO systems and JWT token generation.

Method	Parameters	Return Type	Description
login	cardId: string, password: string	`Promise<{token: string, user: User}>`	Authenticates user via HCMUT SSO and returns JWT token with user data

Table 5: AuthService Class Methods

4.2.2 BillingService Class

The `BillingService` is responsible for calculating parking fees based on various policies, durations, and user types.

Method	Parameters	Return Type	Description
calculateFee	startTime: Date, endTime: Date, vehicleType: string, userRole: string	Promise<number>	Calculates parking fee based on duration, vehicle type, and user role using pricing policies
calculateCycleFee	sessions: ParkingSession[]	Promise<number>	Calculates total fee for multiple parking sessions in a billing cycle

Table 6: BillingService Class Methods

4.2.3 NavigationService Class

The `NavigationService` provides real-time data about parking zone utilization and system-wide statistics to guide users.

Method	Parameters	Return Type	Description
getZonesByUsage	-	Promise<Zone[]>	Retrieves zones sorted by current utilization rate
getStats	-	`Promise<{totalCapacity: number, totalUsage: number, utilizationRate: number}>`	Returns system-wide parking statistics and utilization metrics

Table 7: NavigationService Class Methods

4.2.4 ReportService Class

The `ReportService` generates business intelligence reports, including revenue trends and system health patterns.

Method	Parameters	Return Type	Description
getDailyRevenue	startDate: Date, endDate: Date	Promise<DailyRevenueReport[]>	Generates daily revenue reports for specified date range
getActivityStats	-	`Promise<{usageByHour: number[], avgOccupancyRate: number, deviceErrors: number}>`	Provides hourly usage patterns and system health metrics

Table 8: ReportService Class Methods

4.2.5 SystemAdminService Class

The `SystemAdminService` provides administrative functions for configuring the system and viewing security audit logs.

Method	Parameters	Return Type	Description
getAllConfigs	-	Promise<SystemConfig[]>	Retrieves all system configuration settings
updateConfig	key: string, value: any	Promise<SystemConfig>	Updates a specific system configuration setting
updatePricing	policy: PricingPolicy	Promise<PricingPolicy>	Creates or updates pricing policy rules
getPricingPolicies	-	Promise<PricingPolicy[]>	Retrieves all active pricing policies
getAuditLogs	filter: AuditFilter	Promise<AuditLog[]>	Returns audit logs based on specified filters

Table 9: SystemAdminService Class Methods

4.2.6 SystemLogService Class

The `SystemLogService` handles operational logging across the platform, supporting real-time event monitoring.

Method	Parameters	Return Type	Description
log	level: LogLevel, source: string, message: string, metadata?: object	Promise<void>	Logs system events with specified level and context
getRecent	limit: number	Promise<SystemLog[]>	Retrieves most recent log entries up to specified limit

Table 10: SystemLogService Class Methods

4.3 Controller Layer Methods

4.3.1 EntryExitController Class

The `EntryExitController` orchestrates the physical entry and exit of vehicles by validating permissions and creating sessions.

Method	Parameters	Return Type	Description
checkIn	req: Request, res: Response	Promise<Response>	Processes vehicle entry, validates permissions, creates parking session
checkOut	req: Request, res: Response	Promise<Response>	Processes vehicle exit, calculates fees, updates session status
openGate	req: Request, res: Response	Promise<Response>	Controls physical gate operations for authorized access

Table 11: EntryExitController Class Methods

4.3.2 PaymentController Class

The `PaymentController` handles financial interactions, including billing cycles and debt retrieval for users.

Method	Parameters	Return Type	Description
initiateCyclePayment	req: Request, res: Response	Promise<Response>	Initiates billing cycle payment process for accumulated sessions
mockBKPayCallback	req: Request, res: Response	Promise<Response>	Handles payment gateway callbacks and updates transaction status
getHistory	req: Request, res: Response	Promise<Response>	Returns user's payment history and transaction records
getHistoryAdmin	req: Request, res: Response	Promise<Response>	Administrative view of all payment transactions
getDebt	req: Request, res: Response	Promise<Response>	Retrieves outstanding payment amounts for user

Table 12: PaymentController Class Methods

4.3.3 SessionController Class

The `SessionController` provides endpoints for managing and retrieving parking session data.

Method	Parameters	Return Type	Description
getAll	req: Request, res: Response	Promise<Response>	Retrieves all parking sessions with filtering options
getByUser	req: Request, res: Response	Promise<Response>	Returns parking sessions for specific user
getRecent	req: Request, res: Response	Promise<Response>	Provides recent parking session activity

Table 13: SessionController Class Methods

4.4 Frontend Component Methods

4.4.1 PlacedDevice Class

No public methods defined - data structure for UI mapping.

4.4.2 CanvasTransform Class

No public methods defined - data structure for map transformations.

4.4.3 ParkingMap Class

No public methods defined - data structure for complete map configuration.

4.5 Supporting Class Methods

4.5.1 MappingConfig Class

No public methods defined - configuration data container.

4.5.2 InfrastructureAlert Class

No public methods defined - alert data container.

4.5.3 SystemConfig Class

No public methods defined - configuration data container.

4.5.4 AuditLog Class

No public methods defined - audit data container.

4.5.5 SystemLog Class

No public methods defined - log data container with TTL indexing.

4.5.6 TemporaryCard Class

No public methods defined - temporary access data container.

4.5.7 Location Class

No public methods defined - geographic data container.

4.5.8 Invoice Class

No public methods defined - financial transaction data container.

5 Test Cases

This section documents the unit test cases implemented for the Smart Parking IoT system. Tests are organized by component type and cover critical functionality.

5.1 Backend Unit Tests

5.1.1 Authentication Middleware Tests

ID	Test Scenario	Expected Result	Test Data
AUTH-001	OPTIONS preflight request	Passes without token validation	`method: 'OPTIONS'`
AUTH-002	Missing authorization header	Returns 401 Unauthorized	No Authorization header
AUTH-003	Invalid JWT token	Returns 401 Unauthorized	Invalid JWT string
AUTH-004	Valid JWT token	Calls next() middleware	Valid JWT with correct signature
AUTH-005	Expired JWT token	Returns 401 Unauthorized	Expired JWT token

Table 14: Authentication Middleware Test Cases

5.1.2 User Model Tests

ID	Test Scenario	Expected Result	Test Data
USER-001	Password comparison with correct password	Returns true	Valid password string
USER-002	Password comparison with incorrect password	Returns false	Wrong password string
USER-003	Password hashing during user creation	Password stored as hash	Plain text password

Table 15: User Model Test Cases

5.1.3 Card Controller Tests

ID	Test Scenario	Expected Result	Test Data
CARD-001	Valid card ID validation	Returns user data	Valid school card ID
CARD-002	Invalid card ID validation	Returns 404 error	Non-existent card ID
CARD-003	Card ID format validation	Returns 400 error	Malformed card ID

Table 16: Card Controller Test Cases

5.1.4 Session Controller Tests

ID	Test Scenario	Expected Result	Test Data
SESS-001	Retrieve user sessions	Returns session array	Valid user ID
SESS-002	Filter sessions by status	Returns filtered results	Status: 'ACTIVE'
SESS-003	Session pagination	Returns paginated results	Page: 1, limit: 10
SESS-004	Invalid user ID	Returns 404 error	Non-existent user ID

Table 17: Session Controller Test Cases

5.2 Frontend Unit Tests

5.2.1 API Service Tests

ID	Test Scenario	Expected Result	Test Data
API-001	Successful API call	Returns response data	Valid endpoint and data
API-002	API call with authentication error	Throws authentication error	Invalid JWT token
API-003	Network error handling	Throws network error	Network failure simulation
API-004	API timeout handling	Throws timeout error	Delayed response

Table 18: API Service Test Cases

5.2.2 Monitoring Service Tests

ID	Test Scenario	Expected Result	Test Data
MON-001	Real-time data subscription	Receives data updates	Valid WebSocket connection
MON-002	Connection error handling	Reconnects automatically	Network interruption
MON-003	Data parsing validation	Parses data correctly	Valid monitoring data format
MON-004	Invalid data handling	Ignores invalid data	Malformed data packet

Table 19: Monitoring Service Test Cases

5.2.3 Pricing Policy Service Tests

ID	Test Scenario	Expected Result	Test Data
PRICE-001	Fee calculation for student	Returns discounted fee	Student role, 2 hours
PRICE-002	Fee calculation for faculty	Returns standard fee	Faculty role, 2 hours
PRICE-003	Weekend pricing application	Applies weekend discount	Weekend timestamp
PRICE-004	Invalid role handling	Throws validation error	Unknown user role
PRICE-005	Negative duration handling	Throws validation error	Negative parking duration

Table 20: Pricing Policy Service Test Cases

5.3 Integration Test Scenarios

5.3.1 Authentication Flow Integration

ID	Test Flow	Expected Outcome
INT-AUTH-001	User login → JWT generation → API access	Successful authentication and authorization
INT-AUTH-002	Invalid credentials → Error response	Proper error handling without information leakage
INT-AUTH-003	Token expiration → Automatic refresh	Seamless token renewal

Table 21: Authentication Integration Test Scenarios

5.3.2 Parking Session Flow Integration

ID	Test Flow	Expected Outcome
INT-PARK-001	Check-in → Session creation → Zone update	Successful session initiation with capacity update
INT-PARK-002	Check-out → Fee calculation → Payment processing	Complete session closure with billing
INT-PARK-003	Concurrent check-ins → Capacity limits	Proper queue handling and rejection when full

Table 22: Parking Session Integration Test Scenarios

5.3.3 Payment Flow Integration

ID	Test Flow	Expected Outcome
INT-PAY-001	Session accumulation → Billing cycle → BKPay integration	Successful payment processing
INT-PAY-002	Payment failure → Retry mechanism → Notification	Proper error recovery and user notification
INT-PAY-003	Partial payments → Debt tracking → Follow-up	Accurate debt calculation and tracking

Table 23: Payment Integration Test Scenarios

5.4 Test Coverage Metrics

5.4.1 Backend Test Coverage

Component	Statements	Branches	Functions
Controllers	85%	80%	90%
Services	90%	85%	95%
Models	95%	90%	100%
Middlewares	88%	82%	92%
Utilities	92%	87%	96%
Overall	**90%**	**85%**	**95%**

Table 24: Backend Test Coverage Metrics

5.4.2 Frontend Test Coverage

Component	Statements	Branches	Functions
Components	80%	75%	85%
Services	88%	82%	92%
Hooks	85%	80%	90%
Utilities	90%	85%	95%
Overall	**85%**	**80%**	**90%**

Table 25: Frontend Test Coverage Metrics

5.5 Test Execution Results

5.5.1 Automated Test Suite

- **Total Tests:** 127 (Backend: 89, Frontend: 38)
- **Pass Rate:** 98.4%
- **Average Execution Time:** 45 seconds
- **Continuous Integration:** Runs on every push to main branch

5.5.2 Performance Benchmarks

- **Unit Test Execution:** $<$ 30 seconds for full backend suite
- **Integration Tests:** $<$ 60 seconds for critical paths
- **Memory Usage:** $<$ 256MB during test execution
- **Parallel Execution:** Supports up to 4 concurrent test runners

5.6 Quality Assurance Process

5.6.1 Code Quality Gates

1. ESLint passes with zero warnings
2. TypeScript compilation succeeds
3. Test coverage meets minimum thresholds
4. Security vulnerability scan passes
5. Performance benchmarks met

5.6.2 Continuous Integration Pipeline

1. Code formatting check (Prettier)
2. Type checking (TypeScript)
3. Linting (ESLint)
4. Unit tests (Jest/Vitest)
5. Integration tests (Supertest/Testing Library)
6. Build verification (successful compilation)

Team Members & Task

Full Name	ID	Task	Percentage
QUACH GIA BAO	2452141	[PM & SC] PDF Composer, Code composer	100%
DIEC TUAN CHI	2452169	[DEV] Parking Fee and Banking Reconcilation	100%
TO QUOC TAI	2453141	[Techlead & SysAdmin] Access Control and Gate Management	100%
LY TRIEU PHONG	2452958	[DEV] Core DB Models and Seeding	100%
THOAI HAO	2452305	[DEV] User Authentication and Role Management	100%
DOAN QUANG DANG	2352249	[QC] System Admin, Reports and Error handling	100%
NGO QUANG HUNG	2352431	[BA] IoT Webhooks and Infrastructure Alert	100%
DANG TRAN THAI BAO	2452120	[PO] Live monitoring and Dashboard Aggregation	100%